

Ai Diabetes Consultant

By Maged Magdy

Diabetes RAG

Diabetes RAG Document Assistant is an AI-powered application designed to answer diabetes-related questions using information retrieved from a provided knowledge base. By combining semantic search with a Large Language Model (LLM), the system generates context-based answers and displays the relevant source documents. This helps users access educational information more efficiently while providing supporting references for the generated responses.

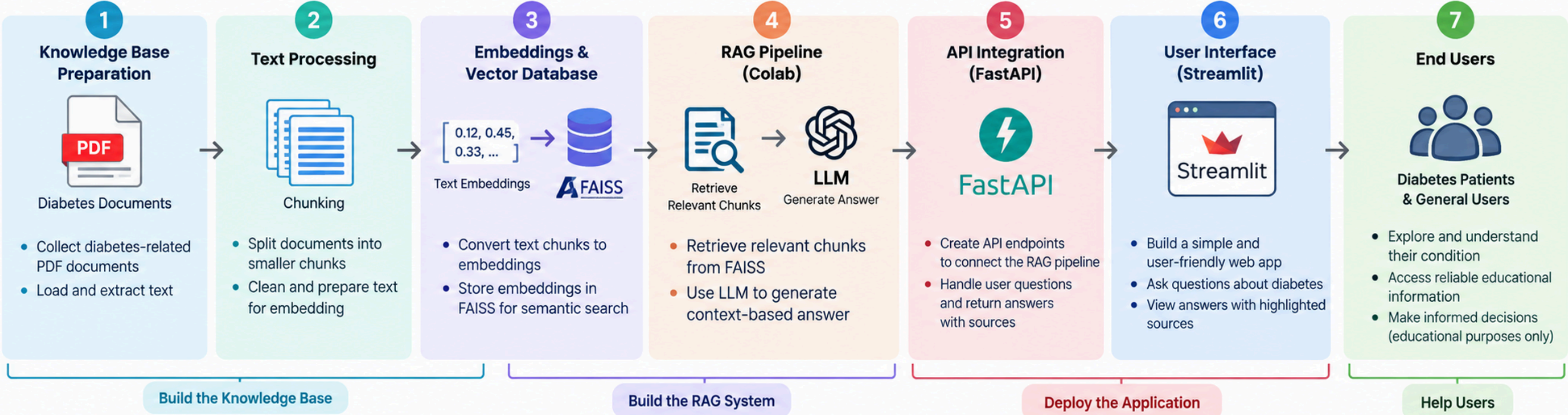
Project Goals

To help diabetes patients explore and better understand their condition through AI-powered, document-based answers with relevant sources, making educational information more accessible and transparent.

Project Roadmap

Diabetes RAG Document Assistant

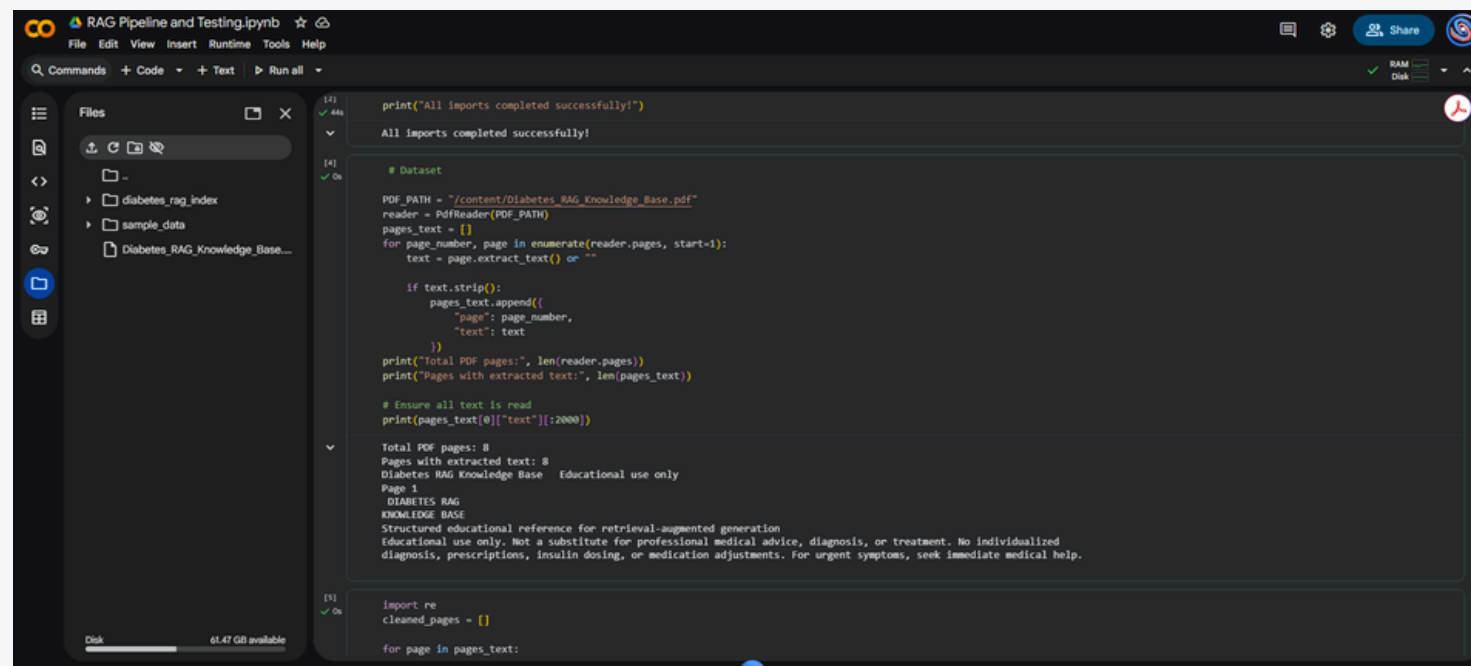
From documents to answers — a step-by-step journey



Our Goal

To help diabetes patients explore and better understand their condition through AI-powered, document-based answers with relevant sources.

Random Photos from the Notebook



```
print("All imports completed successfully!")
All imports completed successfully!

# Dataset
PDF_PATH = "/content/Diabetes_RAG_Knowledge_Base.pdf"
reader = PdfReader(PDF_PATH)
pages_text = []
for page_number, page in enumerate(reader.pages, start=1):
    text = page.extract_text() or ""

    if text.strip():
        pages_text.append({
            "page": page_number,
            "text": text
        })

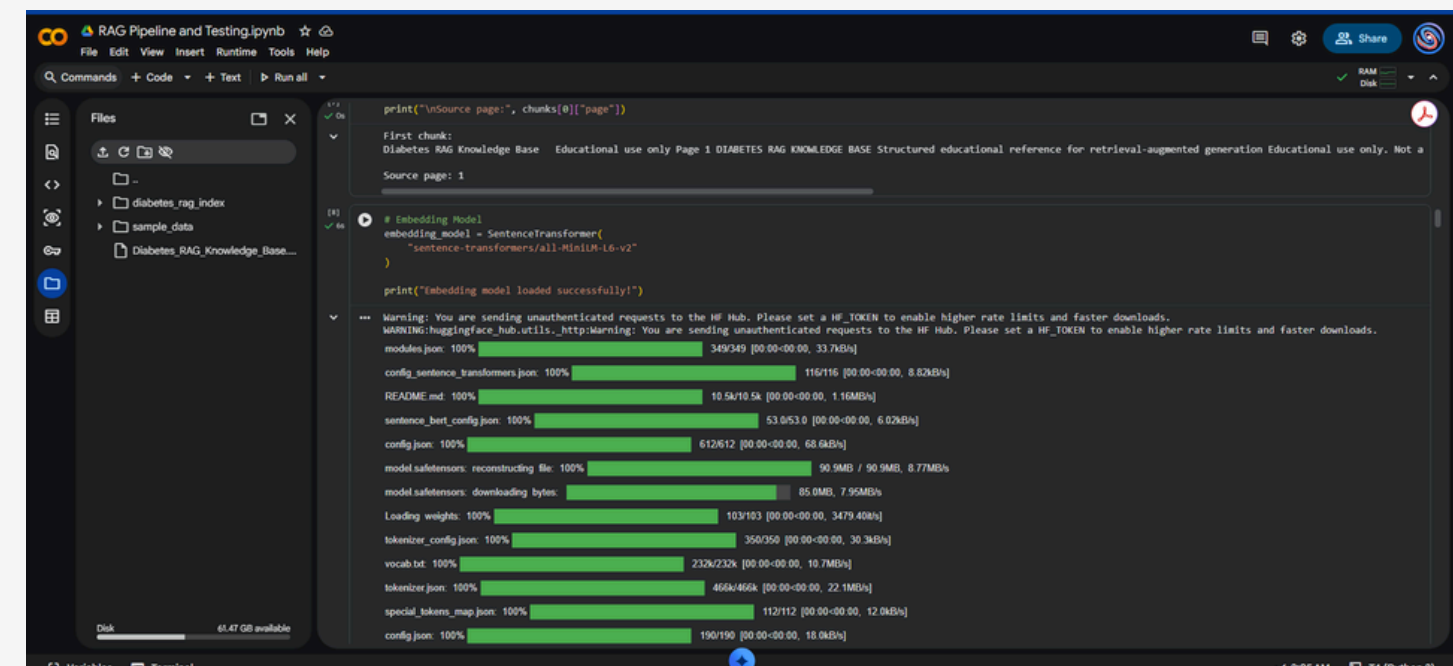
print("Total PDF pages:", len(reader.pages))
print("Pages with extracted text:", len(pages_text))

# Ensure all text is read
print(pages_text[0]["text"][:2000])

Total PDF pages: 8
Pages with extracted text: 8
Diabetes RAG Knowledge Base Educational use only
Page 1
DIABETES RAG
KNOWLEDGE BASE
Structured educational reference for retrieval-augmented generation
Educational use only. Not a substitute for professional medical advice, diagnosis, or treatment. No individualized
diagnosis, prescriptions, insulin dosing, or medication adjustments. For urgent symptoms, seek immediate medical help.

import re
cleaned_pages = []

for page in pages_text:
```

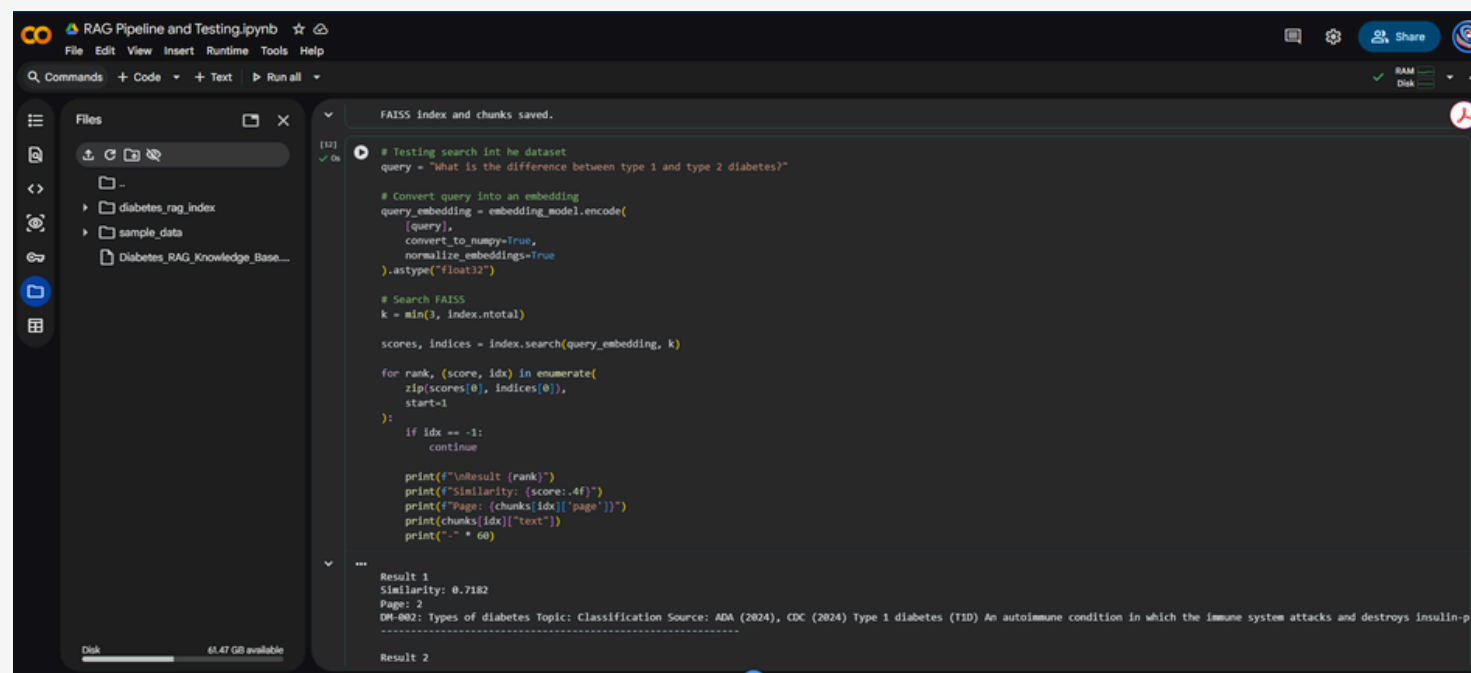


```
print("Source page:", chunks[0]["page"])
First chunk:
Diabetes RAG Knowledge Base Educational use only Page 1 DIABETES RAG KNOWLEDGE BASE Structured educational reference for retrieval-augmented generation Educational use only. Not a
Source page: 1

# Embedding Model
embedding_model = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2"
)

print("Embedding model loaded successfully!")

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
modules.json: 100% 349349 [00:00:00.00, 33.7kB/s]
config_sentence_transformers.json: 100% 116116 [00:00:00.00, 8.82kB/s]
README.md: 100% 10.5k [00:00:00.00, 1.16MB/s]
sentence_bert_config.json: 100% 53.0 [00:00:00.00, 6.02kB/s]
config.json: 100% 612612 [00:00:00.00, 68.6kB/s]
model.safetensors: reconstructing file: 100% 90.9MB / 90.9MB, 8.77MB/s
model.safetensors: downloading bytes: 85.0MB, 7.95MB/s
Loading weights: 100% 103103 [00:00:00.00, 3479.40kB/s]
tokenizer_config.json: 100% 350/350 [00:00:00.00, 30.3kB/s]
vocab.txt: 100% 232k/232k [00:00:00.00, 10.7MB/s]
tokenizer.json: 100% 466k/466k [00:00:00.00, 22.1MB/s]
special_tokens_map.json: 100% 112/112 [00:00:00.00, 12.0kB/s]
config.json: 100% 190/190 [00:00:00.00, 18.0kB/s]
```



```
FAISS index and chunks saved.

# Testing search int he dataset
query = "What is the difference between type 1 and type 2 diabetes?"

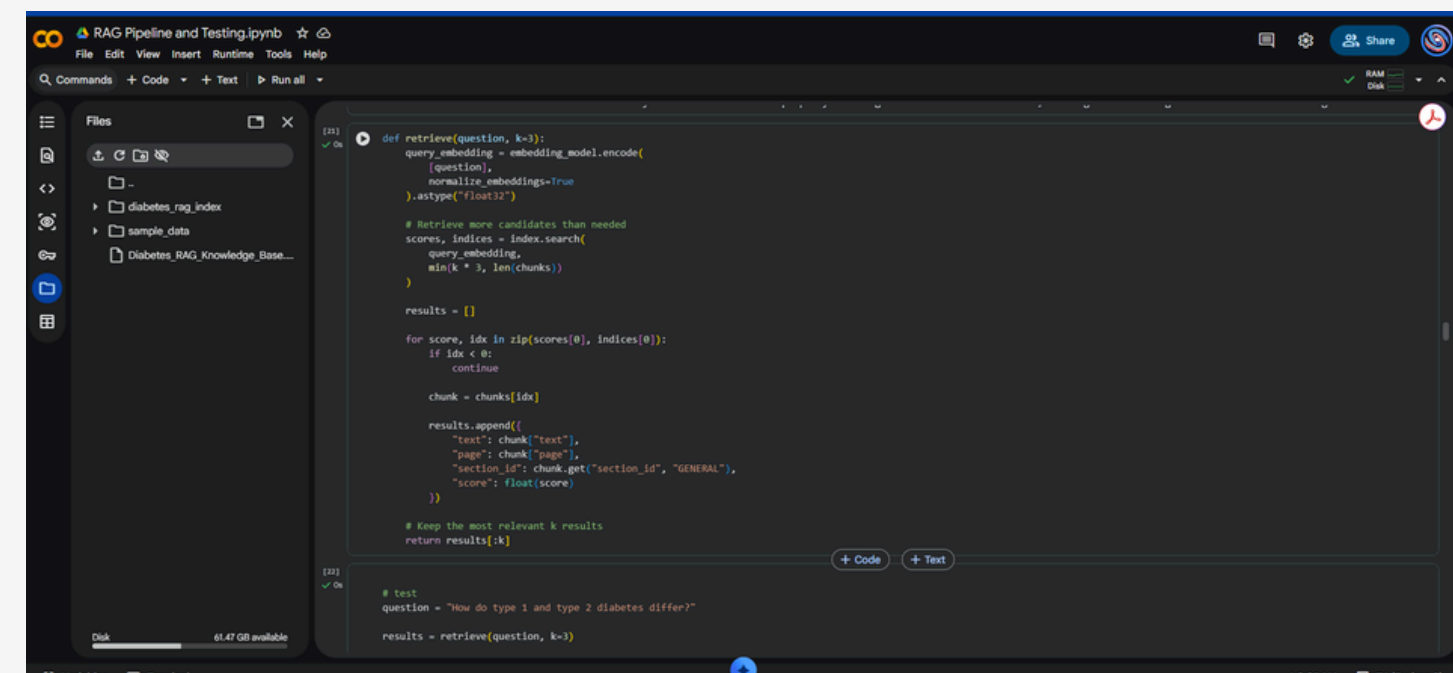
# Convert query into an embedding
query_embedding = embedding_model.encode(
    [query],
    convert_to_numpy=True,
    normalize_embeddings=True
).astype("float32")

# Search FAISS
k = min(3, index.ntotal)
scores, indices = index.search(query_embedding, k)

for rank, (score, idx) in enumerate(
    zip(scores[0], indices[0]),
    start=1
):
    if idx == -1:
        continue

    print(f"Result {rank}")
    print(f"Similarity: {score:.4f}")
    print(f"Page: {chunks[idx]['page']}")
    print(chunks[idx]["text"])
    print("-" * 60)

Result 1
Similarity: 0.7182
Page: 2
DM-002: Types of diabetes Topic: Classification Source: ADA (2024), CDC (2024) Type 1 diabetes (T1D) An autoimmune condition in which the immune system attacks and destroys insulin-p
Result 2
```



```
def retrieve(question, k=3):
    query_embedding = embedding_model.encode(
        [question],
        normalize_embeddings=True
    ).astype("float32")

    # Retrieve more candidates than needed
    scores, indices = index.search(
        query_embedding,
        min(k * 3, len(chunks))
    )

    results = []

    for score, idx in zip(scores[0], indices[0]):
        if idx < 0:
            continue

        chunk = chunks[idx]

        results.append({
            "text": chunk["text"],
            "page": chunk["page"],
            "section_id": chunk.get("section_id", "GENERAL"),
            "score": float(score)
        })

    # Keep the most relevant k results
    return results[:k]

# test
question = "How do type 1 and type 2 diabetes differ?"
results = retrieve(question, k=3)
```

Cmd Launching the Project

```
Command Prompt  Windows PowerShell
INFO: Will watch for changes in these directories: ['C:\\Users\\COMPUMARTS\\OneDrive\\Desktop\\ITI Final Project']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [59724] using WatchFiles
INFO: Started server process [23620]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:58584 - "POST /query HTTP/1.1" 502 Bad Gateway
INFO: 127.0.0.1:58588 - "POST /query HTTP/1.1" 502 Bad Gateway
INFO: 127.0.0.1:58590 - "POST /query HTTP/1.1" 502 Bad Gateway
INFO: 127.0.0.1:58592 - "POST /query HTTP/1.1" 502 Bad Gateway
INFO: 127.0.0.1:58600 - "POST /query HTTP/1.1" 502 Bad Gateway
INFO: 127.0.0.1:63963 - "POST /query HTTP/1.1" 200 OK
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [23620]
INFO: Stopping reloader process [59724]
PS C:\Users\COMPUMARTS\OneDrive\Desktop\ITI Final Project> python -m uvicorn backend.main:app --reload --host 127.0.0.1 --port 8000
INFO: Will watch for changes in these directories: ['C:\\Users\\COMPUMARTS\\OneDrive\\Desktop\\ITI Final Project']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [55932] using WatchFiles
INFO: Started server process [26920]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

```
Command Prompt - streamlit  Windows PowerShell
Local URL: http://localhost:8502
Network URL: http://192.168.1.5:8502

Stopping...

C:\Users\COMPUMARTS\OneDrive\Desktop\ITI Final Project>streamlit run frontend/app.py
2026-09-23 03:00:42.559 Uvicorn server started on ::8502

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8502
Network URL: http://192.168.1.5:8502

Stopping...

C:\Users\COMPUMARTS\OneDrive\Desktop\ITI Final Project>
C:\Users\COMPUMARTS\OneDrive\Desktop\ITI Final Project>streamlit run frontend/app.py
2026-09-23 03:30:16.525 Uvicorn server started on ::8502

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8502
Network URL: http://192.168.1.5:8502
```

Project Interface

 **Diabetes RAG Assistant**

Ask questions about diabetes and get answers based on the provided knowledge base.

Ask your question

What is the difference between diabetes type 1 and diabetes type 2?

Ask

Answer

Diabetes type 1 (T1D) and diabetes type 2 (T2D) differ in the underlying cause of insulin deficiency or resistance.

Sources

> Source 1 | Page 2

> Source 2 | Page 2

> Source 3 | Page 4



Thank You